PROJECT HANDOFF | RESUME WRITING

A11yLens

On-demand accessibility audit browser extension

PROJECT STATUS	VERSION	SOURCE
Local development; evaluation in progress	0.1.0	GitHub main at 05fdcf1

Project summary

A11yLens is a Manifest V3 browser extension that scans a page after the user starts a scan. It runs axe-core locally, separates violations from results that need manual review, highlights affected elements, and connects one selected finding to WCAG guidance through a local FastAPI service. A Groq explanation path is optional and requires a backend API key.

Resume-ready bullets

- Built A11yLens, a Chrome/Brave Manifest V3 extension that runs axe-core 4.14.0 on demand for WCAG 2.0/2.1 A/AA and 2.2 AA tags, separates violations from manual-review findings, and lets users highlight affected elements.
- Developed a local FastAPI guidance API over 55 WCAG 2.2 A/AA summaries with 16 curated axe-rule mappings and TF-IDF retrieval; implemented optional local Sentence Transformers/FAISS retrieval and a Groq explanation path.
- Implemented a local-first data flow with browser session restoration and JSON export that excludes HTML/selectors and strips URL credentials, queries, and fragments; smoke-checked six synthetic rule requests and one popup-to-local-API request.

One-line project entry
A11yLens | Accessibility Audit Browser Extension - JavaScript, HTML/CSS, Chrome Extensions MV3, axe-core, Python, FastAPI, WCAG retrieval, TF-IDF, optional Sentence Transformers/FAISS, optional Groq API.

Use these as drafts. Claude should tailor them to the role and the candidate's actual contribution without adding unsupported metrics or ownership claims.

How the project works

The user starts the scan in the popup. The extension then injects its local scanner into the active page and sends the results back to the popup.

Step	Part	Responsibility
1	Popup	User starts a scan or chooses an action for a finding.
2	Background service worker	Checks the active tab, injects local scanner scripts, and stores the result in browser session storage.
3	Content script + axe-core	Runs the selected rule tags, serializes violations and incomplete results, and highlights requested elements.
4	Popup results	Separates violations and manual-review results, shows count units, and offers inspect/help/guidance actions.
5	FastAPI	Receives one selected finding when the user asks for WCAG guidance, then retrieves references locally.

Browser extension details

- Manifest V3 with `popup.html`, `popup.css`, `popup.js`, `background.js`, and `content.js`. The extension uses the `activeTab`, `scripting`, and `storage` permissions, with one host permission for `http://127.0.0.1:8000/*`.
- Bundles axe-core 4.14.0 locally. Configured tags are `wcag2a`, `wcag2aa`, `wcag21a`, `wcag21aa`, and `wcag22aa`; axe returns `violations` and `incomplete` results.
- Manual-review findings use a neutral Possible issue badge and explanatory text. The UI does not label an incomplete result as a confirmed failure.
- Keeps at most 25 nodes per rule in scan data; each sample HTML string is capped at 2,000 characters and each axe failure summary at 1,200 characters. The total node count is retained separately.
- Users can inspect up to three sample targets, open an axe help link only when it is an HTTPS Deque University URL, and export a JSON report.
- JSON export includes summary data and rule descriptions but omits HTML, selectors, and failure summaries. It removes URL credentials, query parameters, and fragments.

Limit to state clearly
The configured axe tags define the automated checks; the extension is not a WCAG conformance test and cannot find every accessibility problem. Frame, shadow DOM, dynamic-page, and protected-page behavior can also limit scanning or highlighting.

Guidance service and data flow

Backend stack

Python FastAPI exposes `GET /health` and `POST /explain`. Pydantic request fields are strict and length-limited. FastAPI uses `httpx` for the optional Groq request and `python-dotenv` to load the backend's local environment file.

Request field	Maximum length
Axe rule ID	120 characters
Description	500 characters
Impact	30 characters
HTML sample	2,000 characters
Failure summary	1,200 characters
Target selector	500 characters

Retrieval and optional generation

- The local corpus contains 55 WCAG 2.2 A/AA summaries: 31 Level A and 24 Level AA. Each entry links to a W3C WAI Understanding page; it is informative guidance, not the full normative WCAG text.
- Sixteen curated axe-rule mappings are applied first. TF-IDF lexical retrieval fills remaining slots, with up to three criteria returned.
- Optional local semantic retrieval uses Sentence Transformers (`all-MiniLM-L6-v2`) and FAISS cosine similarity. TF-IDF remains the fallback if optional packages or the model are unavailable.
- Without `GROQ_API_KEY`, the API uses local guidance. With a key, the selected finding and retrieved references are sent from the local backend to Groq. The configured HTTP timeout is 35 seconds.

Privacy boundaries

- Scanning runs locally after the user starts it. Scan restoration data stays in `chrome.storage.session` during the browser session.
- Only a selected finding is sent to FastAPI after the user asks for guidance. If Groq is configured, that selected context can be sent onward to Groq.
- The Groq key stays in ignored `backend/.env`; it is not placed in extension code. The documented API command binds to `127.0.0.1:8000`.
- There is no hosted A11yLens backend or authentication system. The hosted deployment model and source license remain undecided.

What has been checked

Local guidance smoke check, 30 October 2026

The local API health route reported `status: ok` and `groq_configured: false`. Six synthetic requests completed using TF-IDF plus curated axe-rule mappings. Each response included the axe-tagged reference listed below; extra criteria came from retrieval.

Rule	Returned by A11yLens	Axe tag reference
`color-contrast`	1.4.3, 1.4.1, 1.4.11	1.4.3 (`wcag143`)
`link-name`	2.4.4, 4.1.2, 2.5.3	2.4.4 (`wcag244`), 4.1.2 (`wcag412`)
`aria-required-children`	1.3.1	1.3.1 (`wcag131`)
`target-size`	2.5.8, 1.4.4	2.5.8 (`wcag258`)
`label-content-name-mismatch`	2.5.3, 1.3.5, 1.3.3	2.5.3 (`wcag253`)
`aria-valid-attr-value`	4.1.2	4.1.2 (`wcag412`)

How to describe that result

This smoke check shows that six controlled local guidance requests returned the expected axe-tagged criteria. It does not measure retrieval accuracy on real page findings. Axe tags are a reference for rule-to-criterion agreement, not independent ground truth about whether a page-level finding is correct. A11yLens currently omits axe tags from its scan payload.

Popup check reported by the user

The user confirmed one Get WCAG guidance request worked after FastAPI was restarted. This verifies one popup-to-local-API request, but the rule ID and result-card category were not recorded. Groq was not configured. The user separately reported that the earlier T1 labels worked on penny-juice.com; the later UI follow-ups still need a browser check.

Open evaluation work

- The two local fixture pages have not yet been scanned in Brave; the nine listed conditions are candidate expectations, not scan results.
- No repeatable coverage score, retrieval accuracy score, repair outcome, or performance benchmark has been recorded.
- No hands-on keyboard or screen-reader review has been completed. The checklist exists, but its results are blank.
- Groq mode has not been tested with a configured key. The extension has not been published in the Chrome Web Store.

Resume and interview handoff

Thirty-second project explanation

"A11yLens is a browser extension I am building to make accessibility review easier during development. It runs axe-core locally on a page after the user starts a scan, separates violations from items needing manual review, and lets the developer inspect an affected element. For a selected finding, a local FastAPI service retrieves WCAG guidance. The local path is the one I have smoke-checked; the optional Groq path and hands-on accessibility evaluation are still pending."

Useful interview topics

- Why axe incomplete results need human review instead of a failure label.
- How curated rule mappings and lexical search work together, and why the optional embedding path has a fallback.
- How active-tab access and explicit user actions shape the data flow.
- Why axe WCAG tags can help compare mappings but cannot independently verify a page finding.
- What the next evaluation should record: observed rules, category, affected nodes, human review, and repair outcome.

Skills to list only if interview-ready

JavaScript | Browser Extension APIs / Manifest V3 | axe-core | Python | FastAPI | REST/JSON | WCAG 2.2 | TF-IDF retrieval | FAISS (optional implementation) | Git/GitHub

```
Prompt to give Claude
I want to add A11yLens to my resume. Use this brief as the source of truth and tailor the wording to the resume and job description I provide. Give me a concise project heading, one-sentence summary, two or three action-led bullets, a short version for limited space, and interview talking points. Do not invent usage numbers, accuracy, coverage, time saved, performance, deployment, or sole ownership. Describe the six API requests as a synthetic smoke check, not a formal accuracy evaluation. State that Groq, fixture browser scans, and assistive-technology checks remain untested or pending. Do not claim WCAG compliance, Chrome Web Store publication, a hosted backend, or an A11yLens source license. Ask me which parts I personally implemented before making attribution claims.
```

Source files

For deeper review, share `README.md`, `docs/ARCHITECTURE.md`, `docs/ROADMAP.md`, `docs/PROJECT_LOG.md`, `extension/manifest.json`, `extension/background.js`, `extension/content.js`, `extension/popup.js`, `backend/main.py`, `backend/rag.py`, `backend/data/wcag.json`, and `evaluation/README.md`.

All claims in this brief are bounded by the recorded source and checks. Automated findings are not a WCAG conformance verdict.